UNIVERSITY INSTITUTE of COMPUTING
Asia's Fastest Growing University

NAAC GRADE A+
Accredited University

CHANDIGARH UNIVERSITY
Discover. Learn. Empower.

Mini Project Report

On

Habit Tracker

A Python GUI Application (Tkinter)

SUBMITTED BY:

KARAN KUMAR SAHOO

UID: 26MCA20059

CLASS: 26MCA-GEN-1(B)

SUBJECT: COMPUTING APTITUDE

SUBMITTED TO:

Mr Arshid Ahmad Wani (E15587)

Assistant Professor

Table of Content

S. No.	Name	Page no.
1	Acknowledgement	3
2	Introduction / Background	4
3	Objectives	4
4	Technologies Used	4
5	Requirements and System Design	5
6	Source Code & Output	7
7	Functionalities	14
8	Conclusion / Future Work / Learning Outcomes	15
9	Bibliography	15

Acknowledgement

I would like to express my sincere gratitude to everyone who contributed to the successful completion of this mini project, the Habit Tracker – a desktop application written in Python with a Tkinter graphical interface to help people build and follow daily habits.

First and foremost, I am thankful to my mentor, Mr Arshid Ahmad Wani, Assistant Professor, for his continuous guidance and encouragement throughout the Computing Aptitude course. His insights into problem solving, program design and clean coding practices were invaluable in shaping the direction of this work.

I would also like to thank my classmates and friends for their constructive feedback during testing and debugging. Their suggestions on input validation, the layout of the window and the progress display helped make the program more reliable and easier to use.

Lastly, I am grateful to the Python community and to the authors of the free documentation and tools, such as the Python Standard Library and the Tkinter toolkit, which made it possible to build this project from scratch.

This project stands as a testament to curiosity, practice and perseverance, and I am proud to present it as a step forward in my journey as a programmer.

1. Introduction

This report describes the design and development of the "Habit Tracker", a desktop GUI application written in Python using the Tkinter library. The program lets a user add daily habits, mark each habit as completed (or undo it) for the current day, view the details of a habit such as its creation date, total completed days and current streak, delete habits, and watch a progress bar that shows what percentage of today's habits are done.

All habits are stored as a list of Python dictionaries and saved to a JSON file (habits.json) after every change, so the data is still there the next time the program is opened. At start-up the file is read, checked and, if it is damaged, backed up and replaced by a fresh list. The project demonstrates core programming concepts – lists, dictionaries, functions, loops, file handling with JSON, date arithmetic, exception handling and event-driven GUI programming – in a practical, everyday application.

2. Background

Building a good habit needs consistency, and seeing a streak of unbroken days is a strong motivation to continue. Many habit apps need accounts, an internet connection or a subscription, while a paper diary needs the days to be counted by hand. A small offline desktop tracker that keeps its data in a plain file solves these problems and is also an excellent exercise for learning how real programs build a window, react to button clicks, validate input and store data permanently.

The Habit Tracker was built as a first version of such a tool. It uses only the Python Standard Library (tkinter, json, os and datetime), so it runs on any system with Python 3 (Windows, Linux or macOS) without installing extra packages. Unlike a purely in-memory program, the data is persistent: every change is written immediately to habits.json.

3. Objectives

- Develop a complete habit tracker with a graphical interface using Python and Tkinter.
- Let the user add a habit by typing a name and pressing the Add Habit button or the Enter key.
- Mark a habit as completed or undo it for today, so that a day is recorded only once.
- Show the creation date, total completed days and current streak of each habit.
- Display today's progress as a percentage, a progress bar and a text summary.
- Save data permanently in a JSON file and recover safely if the file is damaged.
- Validate input (empty names, duplicates, no selection) and confirm before deleting.
- Practise modular programming by dividing the logic into small, reusable functions.

4. Technologies Used

- **Language:** Python 3 for the complete program logic.
- **GUI Library:** tkinter and tkinter.ttk (Tk, Label, Entry, Button, Frame, Treeview, Progressbar, Scrollbar, Style) with tkinter.messagebox for dialog boxes.
- **Standard Library:** json for reading and writing data, os for file paths and the backup file, datetime (date, timedelta) for today's date and streaks.
- **Data Storage:** habits are held in memory as a list of dictionaries (habits) and saved to the file habits.json.
- **Tools:** Python interpreter, terminal / command prompt and a code editor such as VS Code, IDLE or PyCharm.
- **Interface:** a 520 x 560 window with a light purple theme, a two-column table, coloured buttons and a green progress bar.

5. Requirements

Software and Hardware Requirements

- Python 3 interpreter with Tkinter (included in the standard Python installers for Windows and macOS).
- A terminal or command prompt (or IDLE) to run the program.
- A code editor or IDE (VS Code, IDLE, PyCharm, etc.) for development.
- Any computer running Windows, Linux or macOS with a display; no special hardware is needed.

Functional Requirements

- Add a habit with a unique name.
- Mark a habit completed for today or undo it.
- View the details and streak of a habit and delete a habit after confirmation.
- Calculate and display today's progress percentage and summary.
- Load habits from habits.json at start-up and save them after every change.

Non-Functional Requirements

- Ease of use through one window, coloured buttons, a selectable list and the Enter key.
- Reliability through input validation, a backup of a damaged data file and visible save errors.
- Portability – uses only the Python Standard Library with no external packages.
- Persistence – data is kept between runs.
- Maintainability – small functions, each with a single responsibility.

6. System Design

Architecture Overview

The application follows a simple three-layer structure inside a single source file. The presentation layer is the Tkinter window (heading, text box, habit table, buttons and progress section) that shows the data and receives the user's clicks. The logic layer contains the functions that add, complete, delete and describe habits and calculate streaks and progress. The data layer is the habits list in memory and the file habits.json that is read at start-up and written after every change. The program is event-driven: after the window is built, root.mainloop() waits, and each click or key press calls one function that updates the list, saves the file and calls refresh_display().

Component Overview

Module	Functions	Purpose
User Interface	root, Label, Entry, Button, Treeview, Progressbar	Window layout and event handling
Data Storage	load_habits(), save_habits(), FILE_NAME	Read / write habits.json, back up a damaged file
Date & Streak Logic	today_string(), is_completed_today(), get_streak()	Today's date, status and consecutive-day streak
Habit Management	add_habit(), toggle_complete(), delete_habit(), view_details()	Add, complete / undo, delete and show details
Display	refresh_display(), get_selected_index()	Redraw list and progress, find the selected row

Data Structure

All habits are kept in one list called habits. Each habit is a dictionary with three keys. Dates are stored as text in the form YYYY-MM-DD so that they can be compared and calculated easily.

Variable	Type	Description
FILE_NAME	str	Full path of habits.json (in the same folder as the script)
habits	list	All habits in memory; every item is a dictionary
habit["name"]	str	Habit name (unique, ignoring upper / lower case)
habit["created"]	str	Date the habit was created, e.g. 2026-10-26
habit["completed_dates"]	list of str	Every date on which the habit was completed
table	ttk.Treeview	Two-column list (Habit, Status); the row id is the list index
progress_bar	ttk.Progressbar	Today's completion percentage (0 to 100)

Record format: habits.json holds one object with the key "habits" whose value is the list of habit dictionaries, for example {"habits": [{"name": "Drink Water", "created": "2026-10-26", "completed_dates": ["2026-10-26"]}]}.

Data Flow

Fig. 1: Start-up flow

Application - data flow (1/2)

```
[Start: python habit_tracker.py]
          |
          v
[Build window: labels, entry, table, buttons, progress]
          |
          v
[load_habits()] --file missing--> [empty list]
          |      --file damaged--> [save habits.json.bak + warning + empty list]
          v
[refresh_display(): fill table, progress bar, date]
          |
          v
[root.mainloop(): wait for a click or key press] <------+
          |                                             |
          v                                             |
[Run the function of the clicked button] --refresh-------+
```

Fig. 2: Operations

Application - data flow (2/2)

```
[Add Habit ] -> name empty or duplicate? -> show warning, stop
             -> append {name, created, completed_dates=[]} -> save_habits()
             -> clear text box -> refresh_display(new row selected)
[Complete/Undo] -> habit selected? -> today in completed_dates?
             -> yes: remove today (undo)   no: append today
             -> save_habits() -> refresh_display()
[View Details ] -> habit selected? -> get_streak() -> message box
[Delete ] -> habit selected? -> ask "Delete ... and its history?"
             -> yes: pop(index) -> save_habits() -> refresh_display()
```

7. Source Code & Output

Source Code (habit_tracker.py)

Snippet 1: Imports, file name and the habits list

habit_tracker.py

```
import json
import os
import tkinter as tk
from datetime import date, timedelta
from tkinter import messagebox, ttk
FILE_NAME = os.path.join(os.path.dirname(os.path.abspath(__file__)), "habits.json")
habits = []
```

Snippet 2: Loading and saving the data file

habit_tracker.py

```
def load_habits():
    """Read habits from habits.json (called once when the program starts)."""
    global habits
    if not os.path.exists(FILE_NAME):
        habits = []          # first run: nothing saved yet
        return
    try:
        with open(FILE_NAME, "r", encoding="utf-8") as file:
            data = json.load(file)
        loaded = []
        for item in data["habits"]:
            # keep only habits that have the fields we need
            if "name" in item and "completed_dates" in item:
                if "created" not in item:
                    item["created"] = today_string()
                loaded.append(item)
        habits = loaded
    except (json.JSONDecodeError, KeyError, TypeError, OSError):
        # The file is damaged: keep a backup copy and start with an empty list
        habits = []
        try:
            os.replace(FILE_NAME, FILE_NAME + ".bak")
        except OSError:
            pass
        messagebox.showwarning("Data problem",
                               "habits.json could not be read.\n"
                               "A backup was saved as habits.json.bak and a new list was started.")


def save_habits():
    """Write all habits to habits.json (called after every change)."""
    try:
        with open(FILE_NAME, "w", encoding="utf-8") as file:
            json.dump({"habits": habits}, file, indent=2)
    except OSError:
        messagebox.showerror("Save error", "Could not save your data to habits.json.")
```

Snippet 3: Date, status and streak helpers

habit_tracker.py

```
def today_string():
    """Today's date as text, e.g. '2026-10-26'."""
    return date.today().isoformat()


def is_completed_today(habit):
    return today_string() in habit["completed_dates"]


def get_streak(habit):
    """Number of days in a row the habit was completed (a pending today does not break it)."""
    dates = habit["completed_dates"]
    day = date.today()
    if day.isoformat() not in dates:
        day = day - timedelta(days=1)       # start counting from yesterday
    streak = 0
```

```
    while day.isoformat() in dates:
        streak = streak + 1
        day = day - timedelta(days=1)
    return streak


def get_selected_index():
    """Return the position of the selected habit in the list, or None if nothing is selected."""
    selection = table.selection()
    if len(selection) == 0:
        messagebox.showinfo("Select a habit", "Please click a habit in the list first.")
        return None
    return int(selection[0])
```

Snippet 4: Redrawing the list and progress section

habit_tracker.py

```
def refresh_display(selected_index=None):
    """Redraw the habit list, progress bar and status message."""
    for row in table.get_children():
        table.delete(row)
    completed = 0
    for index in range(len(habits)):
        habit = habits[index]
        if is_completed_today(habit):
            status = "Completed"
            completed = completed + 1
        else:
            status = "Pending"
        table.insert("", "end", iid=str(index), values=(habit["name"], status))

    # 2. Keep the same row selected (if there is one)
    if selected_index is not None and selected_index < len(habits):
        table.selection_set(str(selected_index))

    # 3. Update the progress section
    total = len(habits)
    if total > 0:
        percent = round(completed / total * 100)
    else:
        percent = 0
    progress_bar["value"] = percent
    percent_label.config(text="Today's progress: " + str(percent) + "%")
    status_label.config(text=str(completed) + " of " + str(total) + " habits completed today")
    date_label.config(text="Date: " + today_string())
```

Snippet 5: Adding a habit and completing / undoing it

habit_tracker.py

```
def add_habit():
    """Add a new habit from the text box."""
    name = name_entry.get().strip()

    if name == "":
        messagebox.showwarning("Empty name", "Please type a habit name.")
        return
    for habit in habits:                    # prevent duplicate names
        if habit["name"].lower() == name.lower():
            messagebox.showwarning("Duplicate", "This habit already exists.")
            return

    habits.append({"name": name, "created": today_string(), "completed_dates": []})
    save_habits()
    name_entry.delete(0, tk.END)
    refresh_display(len(habits) - 1)


def toggle_complete():
    """Mark the selected habit complete for today, or undo it."""
    index = get_selected_index()
    if index is None:
        return
    habit = habits[index]
    today = today_string()
    if today in habit["completed_dates"]:
```

```
        habit["completed_dates"].remove(today)      # undo
    else:
        habit["completed_dates"].append(today)      # only added once per day
    save_habits()
    refresh_display(index)
```

Snippet 6: Deleting a habit and viewing its details

habit_tracker.py

```
def delete_habit():
    """Delete the selected habit after asking for confirmation."""
    index = get_selected_index()
    if index is None:
        return
    name = habits[index]["name"]
    if messagebox.askyesno("Confirm delete", "Delete '" + name + "' and its history?"):
        habits.pop(index)
        save_habits()
        refresh_display()


def view_details():
    """Show details of the selected habit in a message box."""
    index = get_selected_index()
    if index is None:
        return
    habit = habits[index]
    if is_completed_today(habit):
        status = "Completed"
    else:
        status = "Pending"
    streak = get_streak(habit)
    if streak == 1:
        streak_text = "1 day"
    else:
        streak_text = str(streak) + " days"
    text = ("Habit: " + habit["name"] + "\n"
            "Created on: " + habit["created"] + "\n"
            "Today's status: " + status + "\n"
            "Total days completed: " + str(len(habit["completed_dates"])) + "\n"
            "Current streak: " + streak_text)
    messagebox.showinfo("Habit details", text)
```

Snippet 7: Creating the window, style and add-habit row

habit_tracker.py

```
root = tk.Tk()
root.title("Habit Tracker")
root.geometry("520x560")
root.minsize(480, 520)
root.configure(bg="#f3f0ff")           # light purple background

# Style (colours for the progress bar, table and buttons)
style = ttk.Style()
style.theme_use("clam")
style.configure("Green.Horizontal.TProgressbar", troughcolor="#ddd6fe",
                background="#22c55e", thickness=18)
style.configure("Treeview", rowheight=28, font=("Segoe UI", 11))
style.configure("Treeview.Heading", font=("Segoe UI", 11, "bold"))

# Heading
tk.Label(root, text="Habit Tracker", font=("Segoe UI", 22, "bold"),
         bg="#f3f0ff", fg="#5b3fd6").pack(pady=(15, 0))
date_label = tk.Label(root, text="", font=("Segoe UI", 10), bg="#f3f0ff", fg="#6b6890")
date_label.pack()

# Add-habit row: text field + button
add_frame = tk.Frame(root, bg="#f3f0ff")
add_frame.pack(fill="x", padx=20, pady=12)
name_entry = tk.Entry(add_frame, font=("Segoe UI", 12))
name_entry.pack(side="left", fill="x", expand=True, ipady=4)
name_entry.bind("<Return>", lambda event: add_habit())      # Enter key also adds
tk.Button(add_frame, text="Add Habit", command=add_habit, bg="#7c5cff", fg="white",
          font=("Segoe UI", 11, "bold"), relief="flat", padx=12).pack(side="left", padx=(8, 0))
```

Snippet 8: Habit table, buttons, progress section and start-up

habit_tracker.py

```
# Habit list (table with two columns)
list_frame = tk.Frame(root, bg="#f3f0ff")
list_frame.pack(fill="both", expand=True, padx=20)
table = ttk.Treeview(list_frame, columns=("name", "status"), show="headings",
                     selectmode="browse", height=8)
table.heading("name", text="Habit")
table.heading("status", text="Status")
table.column("name", width=300)
table.column("status", width=120, anchor="center")
scrollbar = ttk.Scrollbar(list_frame, orient="vertical", command=table.yview)
table.configure(yscrollcommand=scrollbar.set)
table.pack(side="left", fill="both", expand=True)
scrollbar.pack(side="right", fill="y")

# Action buttons
button_frame = tk.Frame(root, bg="#f3f0ff")
button_frame.pack(pady=12)
tk.Button(button_frame, text="Complete / Undo", command=toggle_complete, bg="#22c55e", fg="white",
          font=("Segoe UI", 10, "bold"), relief="flat", padx=10, pady=4).pack(side="left", padx=4)
tk.Button(button_frame, text="View Details", command=view_details, bg="#3b82f6", fg="white",
          font=("Segoe UI", 10, "bold"), relief="flat", padx=10, pady=4).pack(side="left", padx=4)
tk.Button(button_frame, text="Delete", command=delete_habit, bg="#ef4444", fg="white",
          font=("Segoe UI", 10, "bold"), relief="flat", padx=10, pady=4).pack(side="left", padx=4)

# Progress section
progress_frame = tk.Frame(root, bg="#ffffff", padx=15, pady=10)
progress_frame.pack(fill="x", padx=20, pady=(0, 15))
percent_label = tk.Label(progress_frame, text="", font=("Segoe UI", 11, "bold"), bg="#ffffff")
percent_label.pack(anchor="w")
progress_bar = ttk.Progressbar(progress_frame, style="Green.Horizontal.TProgressbar",
                               length=400, maximum=100, mode="determinate")
progress_bar.pack(fill="x", pady=6)
status_label = tk.Label(progress_frame, text="", font=("Segoe UI", 10), bg="#ffffff", fg="#6b6890")
status_label.pack(anchor="w")


load_habits()
refresh_display()
name_entry.focus()
root.mainloop()
```

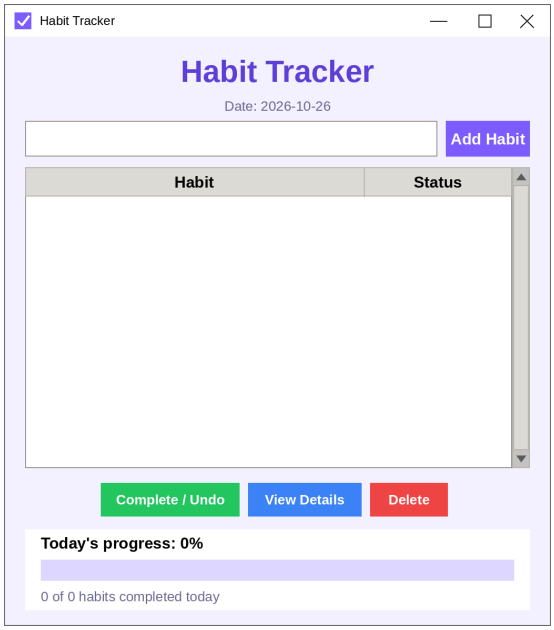
Compilation and Execution

Terminal - bash

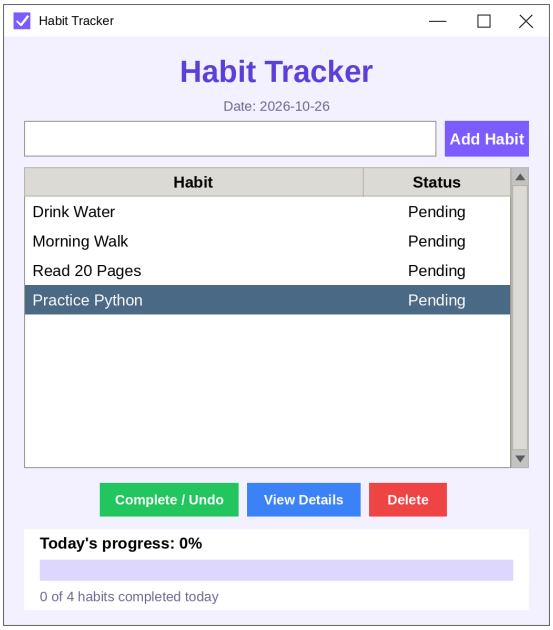
```
$ cd HabitTracker
$ python habit_tracker.py      # Windows: py habit_tracker.py
```

Output

Output 1: Start-up with an empty list, and the list after adding four habits

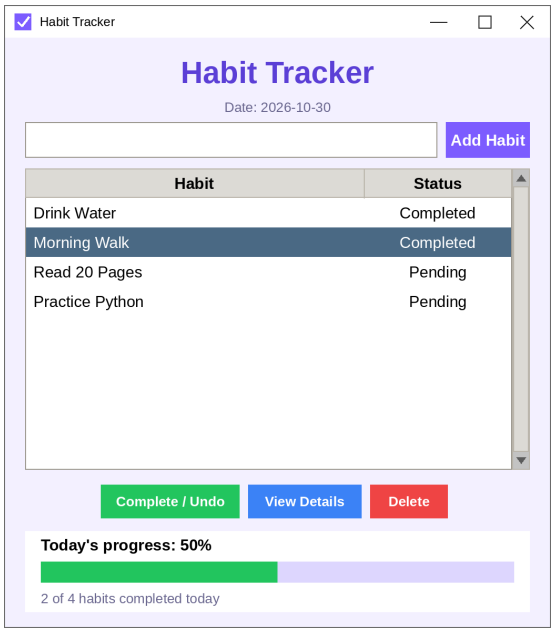


(a) First run: no habits, progress 0%

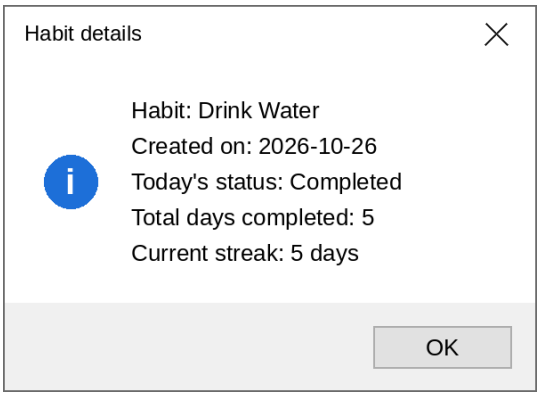


(b) Four habits added; the new row is selected

Output 2: Marking habits as completed (progress 50%) and viewing the details of a habit

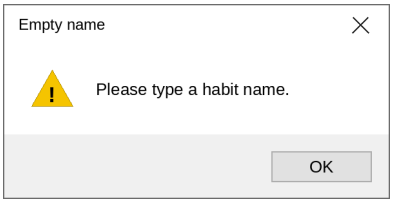


(a) Two of four habits completed today

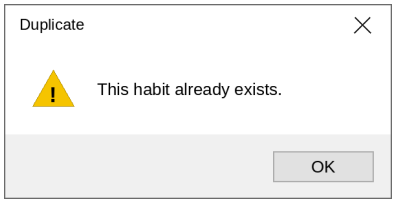


(b) View Details message box

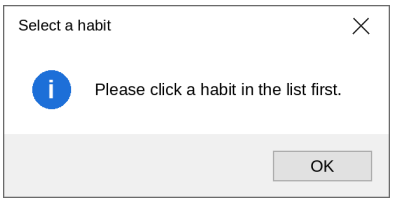
Output 3: Input validation messages



(a) Empty name

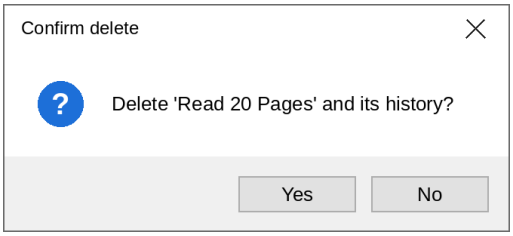


(b) Duplicate habit

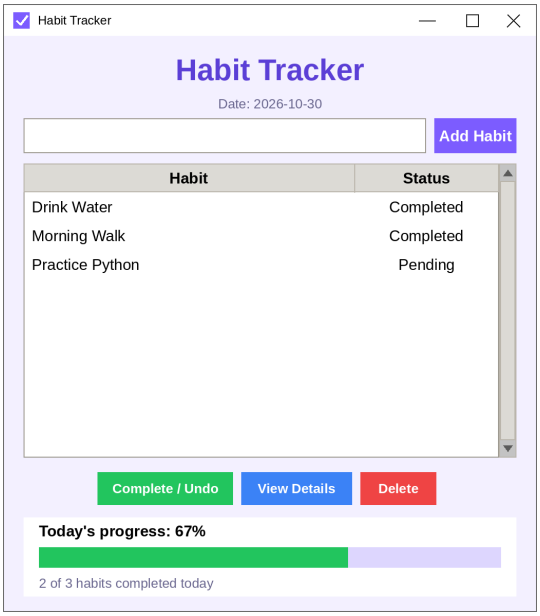


(c) Nothing selected

Output 4: Deleting a habit with confirmation, and the list after deleting

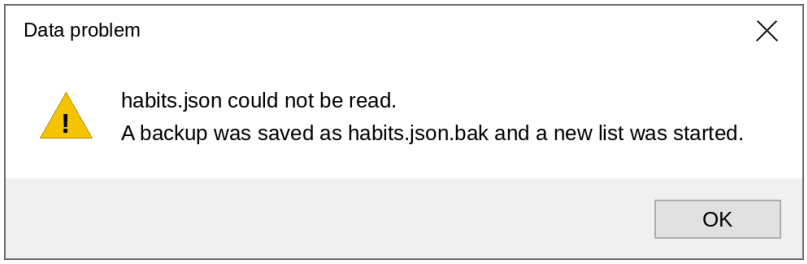


(a) Confirmation box



(b) 'Read 20 Pages' removed; progress 2 of 3 = 67%

Output 5: Damaged data file warning and the saved habits.json



(a) Shown when habits.json cannot be read (a backup habits.json.bak is created)

habits.json

```
{
  "habits": [
    {
      "name": "Drink Water",
      "created": "2026-10-26",
      "completed_dates": [
        "2026-10-26",
        "2026-10-27",
        "2026-10-28",
        "2026-10-29",
        "2026-10-30"
      ]
    },
    {
      "name": "Morning Walk",
      "created": "2026-10-26",
      "completed_dates": [
        "2026-10-28",
        "2026-10-29",
        "2026-10-30"
      ]
    },
    {
      "name": "Practice Python",
      "created": "2026-10-26",
      "completed_dates": []
    }
  ]
}
```

(b) Contents of habits.json written by save_habits() with indent=2

8. Functionalities

Add Habit

- Reads the name typed in the text box (extra spaces are removed) when Add Habit is clicked or Enter is pressed.
- Rejects an empty name and a duplicate name (upper / lower case is ignored), then saves immediately and selects the new row.

Complete / Undo

- Works on the selected habit; if nothing is selected a message asks the user to click a habit first.
- Adds today's date to completed_dates, or removes it if it is already there, so a habit is counted only once per day.

View Details

- Shows the habit name, creation date, today's status, total days completed and the current streak ("1 day" or "n days").
- The streak counts consecutive completed days ending today; if today is still pending it starts from yesterday, so a pending day does not break the streak.

Delete

- Asks for confirmation ("Delete ... and its history?") and then removes the habit and all its dates from the list and from the file.

Progress

- Progress = round(completed / total x 100), or 0 when there are no habits; it is shown as a label, a green progress bar and the text "x of y habits completed today", together with the current date.

Controls Summary

Control	Action	Messages shown
Add Habit / Enter key	Adds a new habit and saves	Empty name, Duplicate
Complete / Undo	Toggles today's status	Select a habit
View Details	Shows details and streak	Select a habit
Delete	Removes the habit after confirmation	Select a habit, Confirm delete

Robustness Features

- **Missing file:** on the first run habits.json does not exist, so the program simply starts with an empty list.
- **Damaged file:** if the JSON cannot be read, the file is renamed to habits.json.bak, a warning is shown and a new list is started.
- **Incomplete records:** habits without a name or completed_dates are skipped; a missing created date is replaced by today's date.
- **Save errors:** if the file cannot be written, an error box is shown instead of the program crashing.

9. Conclusion

The Habit Tracker successfully meets its goals. It provides a simple, reliable way to build daily habits, see today's progress and streaks, and keep the data safely between sessions. Input validation and careful file handling ensure that wrong input or a damaged data file never crashes the program. Built entirely with the Python Standard Library, the project shows how a small desktop application can solve a real everyday problem while practising important programming concepts.

10. Future Work

- **Edit Habits:** rename a habit and set its target days or reminder time.
- **Reminders:** show a notification when a habit is still pending.
- **Database:** store the data in SQLite instead of a JSON file for faster searching and larger histories.
- **Statistics:** weekly and monthly charts, a calendar view and the best streak of each habit.
- **Export:** save the history to a CSV file for use in a spreadsheet.
- **Themes:** add a dark mode and a mobile or web version.

11. Learning Outcomes

- Gained practical experience with lists, dictionaries and strings in Python.
- Learned to read and write JSON files and to handle errors with try / except.
- Learned to work with dates using date and timedelta to calculate streaks.
- Built a GUI with Tkinter and ttk: pack layout, Entry, Button, Treeview, Progressbar, key binding and message boxes.
- Improved skills in modular design, breaking a problem into small reusable functions.

12. Bibliography

- Python Software Foundation. (n.d.). tkinter – Python interface to Tcl/Tk. Retrieved from https://docs.python.org/3/library/tkinter.html
- Python Software Foundation. (n.d.). tkinter.ttk – Tk themed widgets. Retrieved from https://docs.python.org/3/library/tkinter.ttk.html
- Python Software Foundation. (n.d.). json – JSON encoder and decoder. Retrieved from https://docs.python.org/3/library/json.html
- Python Software Foundation. (n.d.). datetime – Basic date and time types. Retrieved from https://docs.python.org/3/library/datetime.html
- TkDocs. (n.d.). Modern Tk best practices and tutorial. Retrieved from https://tkdocs.com/